TECNOLÓGICO DE ESTUDIOS SUPERIORES DE ECATEPEC

DIVISIÓN DE INGENIERÍA EN SISTEMAS COMPUTACIONALES

PROGRAMACIÓN DE BASE DE DATOS

TAREA 10: PROGRAMACIÓN DEL LADO DEL CLIENTE: SQL EMBEBIDO CON C

PROFESORA:

ALEJANDRA GONZALEZ SÁNCHEZ

NOMBRE:

MONTERRUBIO LÓPEZ LUIS GAEL

GRUPO: 5701

FECHA: 30-10-26

1. Introducción y objetivo

Este informe explica la teoría del SQL embebido en C —cómo se conecta un programa a la base de datos, cómo ejecuta las consultas y cómo maneja los resultados— y describe el desarrollo de un módulo para el proyecto IoT Analytics Dashboard de NexoDev Solutions: un programa en C que gestiona los datos de los sensores IoT en PostgreSQL mediante operaciones CRUD.

El programa, `iot_crud`, usa **ECPG**, el precompilador de SQL embebido de PostgreSQL. Las sentencias SQL se escriben directamente dentro del código C con la forma `EXEC SQL ...`, y antes de compilar, ECPG las traduce a llamadas de su biblioteca. Con él se crean, consultan, actualizan y eliminan sensores y lecturas; además, el mismo programa funciona como simulador que almacena lecturas en tiempo real.

Repositorio del proyecto: https://github.com/luisgael-ml/iot-sql-c

Competencia específica: manejar un lenguaje de programación visual y la conexión a la base de datos mediante código SQL embebido (clave de actividad BDM-1301T03CE01A07).

Objetivos específicos:

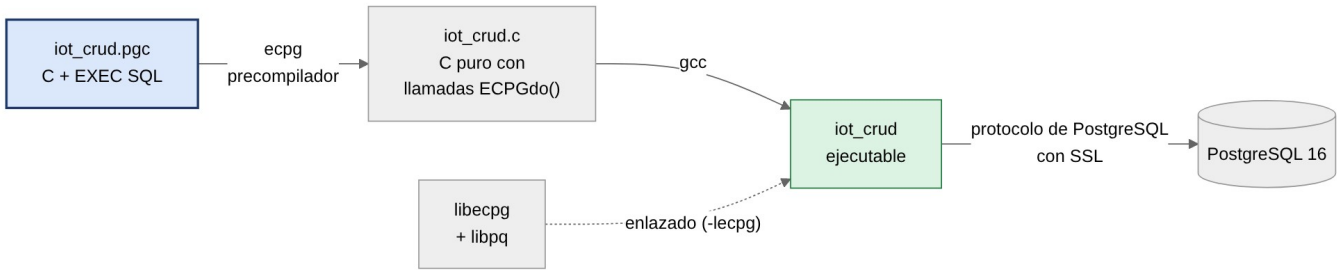
1. Diseñar la estructura de la base de datos en PostgreSQL para almacenar datos de sensores IoT.
2. Configurar un entorno de desarrollo con Docker para la base de datos y para el programa en C.
3. Escribir el código SQL embebido en C para realizar operaciones CRUD.
4. Probar el programa con datos simulados de sensores IoT y verificar el funcionamiento de cada operación.
5. Documentar el proceso de desarrollo y los resultados, con el código fuente y ejemplos de uso.

2. Marco teórico: SQL embebido en C

2.1 ¿Qué es el SQL embebido?

El SQL embebido es una técnica en la que las sentencias SQL se escriben **dentro del código fuente de un lenguaje anfitrión**, en este caso C. El estándar SQL (ISO/IEC 9075) define su sintaxis: cada sentencia comienza con `EXEC SQL` y termina con punto y coma, y puede usar variables del programa precedidas de dos puntos (`:variable`).

Como el compilador de C no entiende SQL, se usa un **precompilador**. En PostgreSQL es ECPG: lee el archivo `.pgc`, reemplaza cada `EXEC SQL` por llamadas a la biblioteca `libecpg` y genera un archivo `.c` normal, que después compila gcc. En tiempo de ejecución, `libecpg` usa `libpq` para hablar con el servidor.



Del código con SQL embebido al ejecutable conectado a PostgreSQL.

Por ejemplo, esta línea del programa:

```
EXEC SQL UPDATE dispositivos SET estado = :estado WHERE codigo = :codigo;
```

ECPG la convierte en esta llamada, donde las variables de C se pasan como parámetros `$1` y `$2`, nunca como texto pegado al SQL, y agrega la revisión de errores declarada con `WHENEVER`:

```
{ ECPGdo(__LINE__, 0, 1, NULL, 0, ECPGst_normal,
         "update dispositivos set estado = $1  where codigo = $2 ",
         ECPGt_char,(estado),(long)16,(long)1,(16)*sizeof(char),
         ECPGt_NO_INDICATOR, NULL , 0L, 0L, 0L,
         ECPGt_char,(codigo),(long)21,(long)1,(21)*sizeof(char),
         ECPGt_NO_INDICATOR, NULL , 0L, 0L, 0L, ECPGt_EOIT, ECPGt_EORT);
if (sqlca.sqlcode < 0) manejar_error ( );}
```

2.2 SQL embebido y otras formas de conectarse

ASPECTO	SQL EMBEBIDO (ECPG)	API DE LLAMADAS (LIBPQ)	CONTROLADORES (JDBC, ADO.NET)
Cómo se escribe	`EXEC SQL` dentro de C	Funciones `PQexec`, `PQgetvalue`	Objetos y métodos del lenguaje
Paso previo	Precompilar con ecpg	Ninguno	Ninguno
Variables	Se enlazan automáticamente (`:var`)	Se convierten a texto a mano	Parámetros con `?` o `@nombre`
Errores de sintaxis SQL	Algunos se detectan al precompilar	Solo al ejecutar	Solo al ejecutar
Uso en el curso	Esta tarea (C)	—	Tareas 3 (Java) y 4 (C#)

2.3 Conceptos clave

CONCEPTO	SINTAXIS	PARA QUÉ SIRVE	EN EL PROGRAMA
Sección de declaración	`EXEC SQL BEGIN/END DECLARE SECTION`	Declara las variables de enlace que se intercambian con el servidor	Todas las funciones
Variable de enlace (host variable)	`:codigo`, `:valor`	Lleva datos de C al SQL (entrada) o del SQL a C (salida)	`WHERE s.codigo = :codigo`
Variable indicadora	`:valor :ind_valor`	Indica si el valor es NULL (-1) o si se truncó	Sensores sin lecturas; lecturas sin alerta
Área de comunicación	`EXEC SQL INCLUDE sqlca`	Estructura con el resultado de cada sentencia	`sqlcode`, `sqlstate`, `sqlerrd[2]`
Manejo automático de errores	`EXEC SQL WHENEVER SQLERROR CALL f()`	Ejecuta una función tras cualquier error	`manejar_error()`
Cursor	`DECLARE`, `OPEN`, `FETCH`, `CLOSE`	Recorre una consulta de varias filas, una a la vez	`listar`, `lecturas`, `resumen`
Sentencia preparada	`PREPARE` / `EXECUTE ... USING`	El servidor analiza el SQL una vez y lo ejecuta muchas	`simular`, `benchmark`
SQL dinámico	`EXECUTE IMMEDIATE :texto`	Ejecuta SQL armado en tiempo de ejecución	Forma A del benchmark
Transacciones	`COMMIT`, `ROLLBACK`	Confirman o deshacen el trabajo	Todas las operaciones

2.4 Conexión

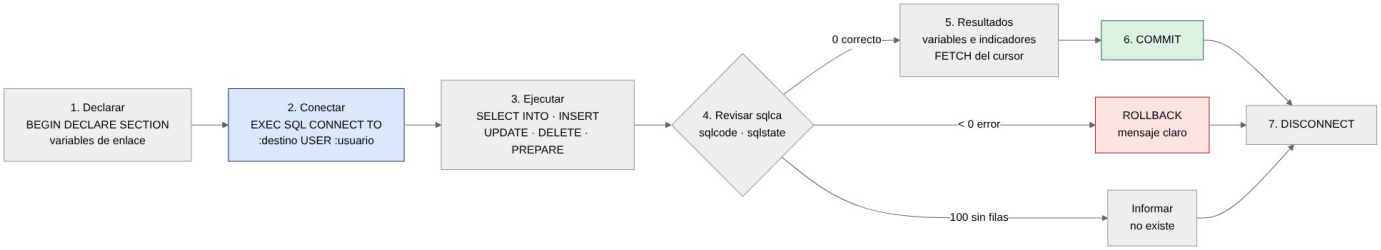
`EXEC SQL CONNECT TO destino AS nombre USER usuario USING clave` abre la conexión. El destino tiene la forma `tcp:postgresql://servidor:puerto/base?opciones`; en el programa se arma con variables de entorno, así la contraseña no queda escrita en el código y se exige SSL con `sslmode=require`. Un programa puede tener varias conexiones con nombre; al final se cierra con `EXEC SQL DISCONNECT`.

2.5 Ejecución de consultas y manejo de resultados

SITUACIÓN	TÉCNICA	RESULTADO EN C
La consulta devuelve una fila	`SELECT ... INTO :v1, :v2`	Los valores quedan en las variables
Devuelve varias filas	Cursor: `FETCH ... INTO` en un ciclo hasta `sqlcode == 100`	Una fila por vuelta
Inserción con dato generado	`INSERT ... RETURNING id INTO :id`	El id asignado por el servidor
Modificación	`UPDATE` / `DELETE`	`sqlca.sqlerrd[2]` = filas afectadas
Valor que puede ser NULL	Variable indicadora	`ind < 0` significa NULL

VALOR DE SQLCA.SQLCODE	SIGNIFICADO	QUÉ HACE EL PROGRAMA
0	Correcto	Continúa
100 (ECPG_NOT_FOUND)	No hubo filas	Informa que el registro no existe
Negativo	Error; `sqlstate` tiene el código estándar de 5 caracteres	Explica el error y hace ROLLBACK

2.6 Ciclo de vida de un programa con SQL embebido



Pasos que sigue cada comando de iot_crud.

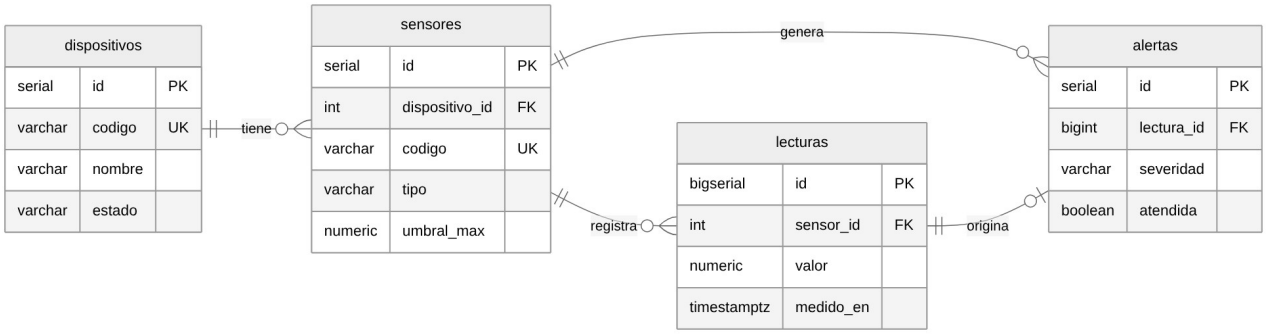
2.7 Ventajas y desventajas

VENTAJAS	DESVENTAJAS
El SQL se lee tal cual dentro del programa	Requiere un paso extra de precompilación
Las variables se enlazan solas, sin convertir tipos a mano	Los errores del código generado son menos claros
Los valores viajan como parámetros: protege contra inyección SQL	Cada gestor tiene su precompilador (ECPG, Pro*C, etc.)
Rendimiento de C: bajo consumo, ideal para equipos IoT y gateways	Menos herramientas y ejemplos que JDBC o un ORM

3. Diseño

3.1 Base de datos

Se usan cuatro tablas relacionadas. Las restricciones (`UNIQUE`, `CHECK`, llaves foráneas) protegen la integridad, y el programa en C traduce sus errores a mensajes claros. Un disparador crea las alertas cuando una lectura supera el umbral, así el programa solo inserta la lectura.



Modelo entidad-relación.

RESTRICCIÓN	EJEMPLO	SQLSTATE SI SE VIOLA
UNIQUE	Código de sensor repetido	23505
CHECK	Tipo no permitido, umbral ≤ 0, estado inválido	23514
FOREIGN KEY	Borrar un sensor que tiene lecturas	23503

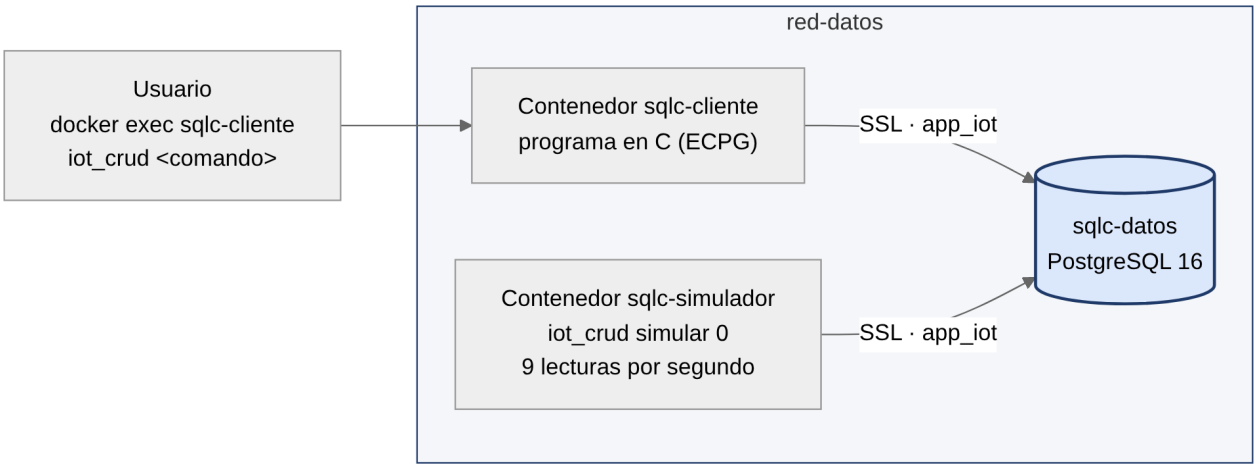
El usuario `app_iot` tiene `SELECT`, `INSERT`, `UPDATE` y `DELETE` sobre las cuatro tablas, pero no puede cambiar su estructura.

3.2 Operaciones CRUD del programa

OPERACIÓN	COMANDO	TÉCNICA DE SQL EMBEBIDO
Crear	`crear-sensor <codigo> <disp> <tipo> <unidad> <umbral>`	`INSERT ... SELECT ... RETURNING INTO`
Crear	`registrar <sensor> <valor>`	`INSERT ... RETURNING INTO` + consulta de la alerta con indicador
Crear	`simular [segundos]`	Sentencia preparada, una transacción por segundo
Leer	`listar`	Cursor con LATERAL e indicadores para sensores sin lecturas
Leer	`sensor <codigo>`	`SELECT ... INTO` con agregados; NOT FOUND si no existe
Leer	`lecturas <sensor> [n]`	Cursor con variables en WHERE y LIMIT
Leer	`resumen`, `monitor`	Agregados con FILTER y cursor por tipo; actualización cada 2 s
Actualizar	`umbral <sensor> <valor>`	`SELECT ... FOR UPDATE` + `UPDATE`; filas afectadas
Actualizar	`estado <disp> <estado>`	`UPDATE`; CHECK del estado
Eliminar	`eliminar-lectura <id>`	`DELETE` (las alertas se borran en cascada)
Eliminar	`eliminar-sensor <codigo> [--con-lecturas]`	Tres `DELETE` en una sola transacción: todo o nada
Rendimiento	`benchmark [n]`	SQL dinámico contra sentencia preparada; COMMIT por fila contra una transacción

3.3 Entorno con Docker

El programa se compila dentro de Docker con una construcción de dos etapas: la primera instala gcc y ECPG, precompila y compila; la segunda solo contiene el ejecutable y la biblioteca `libecpg`, sin compilador. Así no hace falta instalar nada de C en Windows. El mismo ejecutable corre en dos contenedores: uno para los comandos del usuario y otro como simulador.



Arquitectura del entorno.

4. Implementación

4.1 Estructura del proyecto

```
iot-sql-c/
├── docker-compose.yml        → base de datos, cliente y simulador
├── datos/init.sql            → tablas, restricciones, disparador, usuario y datos
├── cliente/
│   ├── iot_crud.pgc          → programa en C con SQL embebido (ECPG)
│   ├── Makefile              → ecpg + gcc
│   └── Dockerfile            → compilación en dos etapas
└── docs/Documentacion_Tarea10.pdf
```

4.2 Compilación

```
# 1) ecpg traduce el SQL embebido (.pgc) a C puro (.c)
# 2) gcc compila el .c y lo enlaza con la biblioteca libecpg
CC      = gcc
CFLAGS  = -Wall -O2 -I/usr/include/postgresql
LDLIBS  = -lecpg

iot_crud: iot_crud.c
	$(CC) $(CFLAGS) -o $@ $< $(LDLIBS)

iot_crud.c: iot_crud.pgc
	ecpg -o $@ $<

clean:
	rm -f iot_crud iot_crud.c
```

```
# ---------- Etapa 1: precompilar y compilar el programa en C ----------
FROM debian:bookworm-slim AS compilacion
RUN apt-get update \
 && apt-get install -y --no-install-recommends gcc make libc6-dev libecpg-dev \
 && rm -rf /var/lib/apt/lists/*
WORKDIR /src
```

```
COPY Makefile iot_crud.pgc ./
RUN make            # ecpg iot_crud.pgc -> iot_crud.c -> gcc -> iot_crud

# ---------- Etapa 2: imagen final solo con el ejecutable ----------
FROM debian:bookworm-slim
RUN apt-get update \
 && apt-get install -y --no-install-recommends libecpg6 \
 && rm -rf /var/lib/apt/lists/*
COPY --from=compilacion /src/iot_crud /usr/local/bin/iot_crud
COPY --from=compilacion /src/iot_crud.c /opt/iot/iot_crud.c
CMD ["sleep", "infinity"]
```

4.3 Programa en C con SQL embebido: iot_crud.pgc

Buenas prácticas aplicadas: las variables de enlace se declaran dentro de cada función que las usa; los textos se copian con `snprintf` para no desbordar los arreglos; cada error se revisa, se explica y se deshace con ROLLBACK; los contadores de `sqlca` se leen antes del COMMIT; y la contraseña se toma de una variable de entorno.

```
/*
 * ========================================================================
 * TAREA 10 · Programacion del lado del cliente: SQL embebido con C
 * IoT Analytics Dashboard (NexoDev Solutions)
 *
 * Programa en C con SQL embebido (ECPG, precompilador de PostgreSQL).
 * Las sentencias EXEC SQL se escriben dentro del codigo C; el comando
 * "ecpg" las traduce a llamadas de la biblioteca libecpg antes de compilar.
 *
 * Operaciones CRUD sobre los datos de los sensores IoT:
 *   C  crear-sensor, registrar, simular
 *   R  listar, sensor, lecturas, resumen, monitor
 *   U  umbral, estado
 *   D  eliminar-lectura, eliminar-sensor
 *   +  benchmark (analisis de rendimiento)
 * ========================================================================
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

EXEC SQL INCLUDE sqlca;          /* area de comunicacion: codigo y mensaje de cada sentencia */

/* ------------------------------------------------------------------ */
/* Manejo de errores                                                   */
/* ------------------------------------------------------------------ */
static int hubo_error = 0;

/* Se llama automaticamente despues de cualquier sentencia que falle */
static void manejar_error(void)
{
    hubo_error = 1;
    fprintf(stderr, "\n[ERROR SQL] SQLSTATE %.5s (codigo %ld)\n  %.*s\n",
            sqlca.sqlstate, sqlca.sqlcode,
            sqlca.sqlerrm.sqlerrml, sqlca.sqlerrm.sqlerrmc);
}

/* Explica en lenguaje claro los SQLSTATE mas comunes */
static const char *explicar(const char *estado)
{
    if (strncmp(estado, "23505", 5) == 0) return "ya existe un registro con ese codigo";
    if (strncmp(estado, "23503", 5) == 0) return "el registro tiene datos que dependen de el";
    if (strncmp(estado, "23514", 5) == 0) return "el valor no cumple una regla (CHECK) de la
tabla";
```

```
    if (strncmp(estado, "22P02", 5) == 0) return "el formato del dato no es valido";
    if (strncmp(estado, "08", 2) == 0)    return "problema de conexion con el servidor";
    return "revisa el mensaje del servidor";
}

/* Revisa el resultado de la ultima sentencia: si fallo, deshace la transaccion */
static int fallo(const char *operacion)
{
    if (sqlca.sqlcode < 0) {
        char estado[6];
        memcpy(estado, sqlca.sqlstate, 5); estado[5] = '\0';
        fprintf(stderr, "No se pudo %s: %s. Se deshizo la transaccion (ROLLBACK).\n",
                operacion, explicar(estado));
        EXEC SQL ROLLBACK;
        return 1;
    }
    return 0;
}

EXEC SQL WHENEVER SQLERROR CALL manejar_error();

/* ------------------------------------------------------------------ */
/* Conexion                                                            */
/* ------------------------------------------------------------------ */
static const char *env(const char *nombre, const char *defecto)
{
    const char *v = getenv(nombre);
    return (v && *v) ? v : defecto;
}

static int conectar(void)
{
    EXEC SQL BEGIN DECLARE SECTION;       /* variables de enlace (host variables) */
    char destino[256];
    char usuario[64];
    char clave[64];
    char ssl[16];
    EXEC SQL END DECLARE SECTION;

    snprintf(destino, sizeof destino, "tcp:postgresql://%s:%s/%s?sslmode=%s",
             env("DB_HOST", "datos"), env("DB_PORT", "5432"),
             env("DB_NAME", "iotanalytics"), env("DB_SSLMODE", "require"));
    snprintf(usuario, sizeof usuario, "%s", env("DB_USER", "app_iot"));
    snprintf(clave, sizeof clave, "%s", env("DB_PASSWORD", ""));

    EXEC SQL CONNECT TO :destino AS iot USER :usuario USING :clave;
    if (sqlca.sqlcode != 0) {
        fprintf(stderr, "No se pudo conectar a %s\n", destino);
        return -1;
    }
    EXEC SQL SELECT CASE WHEN ssl THEN 'SSL activo' ELSE 'sin SSL' END INTO :ssl
             FROM pg_stat_ssl WHERE pid = pg_backend_pid();
    EXEC SQL COMMIT;
    if (getenv("IOT_SILENCIO") == NULL)
        printf("Conectado a PostgreSQL como %s (%s)\n\n", usuario, ssl);
    return 0;
}

/* ================================================================== */
/* R · LEER                                                           */
/* ================================================================== */

/* listar: todos los sensores con su ultima lectura (cursor) */
static int cmd_listar(void)
{
    EXEC SQL BEGIN DECLARE SECTION;
    char codigo[21], tipo[21], unidad[11], disp[21], estado[16], hora[9];
    double umbral, valor;
```

```
    short ind_valor, ind_hora;           /* variables indicadoras: -1 = NULL */
    EXEC SQL END DECLARE SECTION;
    int n = 0;

    EXEC SQL DECLARE cur_sensores CURSOR FOR
        SELECT s.codigo, s.tipo, s.unidad, s.umbral_max, d.codigo, d.estado,
               u.valor, to_char(u.medido_en, 'HH24:MI:SS')
        FROM sensores s
        JOIN dispositivos d ON d.id = s.dispositivo_id
        LEFT JOIN LATERAL (SELECT l.valor, l.medido_en FROM lecturas l
                           WHERE l.sensor_id = s.id
                           ORDER BY l.medido_en DESC LIMIT 1) u ON TRUE
        ORDER BY s.codigo;

    EXEC SQL OPEN cur_sensores;
    if (fallo("abrir el cursor")) return 1;

    printf("%-11s %-12s %-7s %-14s %8s %8s  %-8s\n",
           "SENSOR", "TIPO", "DISP.", "ESTADO", "UMBRAL", "ULTIMO", "HORA");
    printf("-------------------------------------------------------------------------------\n");
    for (;;) {
        EXEC SQL FETCH cur_sensores INTO :codigo, :tipo, :unidad, :umbral, :disp, :estado,
                                         :valor :ind_valor, :hora :ind_hora;
        if (sqlca.sqlcode == ECPG_NOT_FOUND || sqlca.sqlcode < 0) break;
        if (ind_valor < 0)
            printf("%-11s %-12s %-7s %-14s %8.2f %8s  %-8s\n", codigo, tipo, disp, estado, umbral,
"--", "--");
        else
            printf("%-11s %-12s %-7s %-14s %8.2f %8.2f  %-8s %s\n", codigo, tipo, disp, estado,
                   umbral, valor, hora, valor > umbral ? "<- SOBRE UMBRAL" : "");
        n++;
    }
    EXEC SQL CLOSE cur_sensores;
    EXEC SQL COMMIT;
    printf("\n%d sensores\n", n);
    return 0;
}

/* sensor <codigo>: detalle de un sensor (SELECT ... INTO de una sola fila) */
static int cmd_sensor(const char *cod)
{
    EXEC SQL BEGIN DECLARE SECTION;
    char codigo[21], tipo[21], unidad[11], disp[21], nombre[61];
    double umbral, prom, minimo, maximo;
    long lecturas_hora, alertas;
    short ind_prom, ind_min, ind_max;
    EXEC SQL END DECLARE SECTION;

    snprintf(codigo, sizeof codigo, "%s", cod);
    EXEC SQL SELECT s.tipo, s.unidad, s.umbral_max, d.codigo, d.nombre,
                    COUNT(l.id), AVG(l.valor), MIN(l.valor), MAX(l.valor),
                    (SELECT COUNT(*) FROM alertas a WHERE a.sensor_id = s.id AND NOT a.atendida)
             INTO :tipo, :unidad, :umbral, :disp, :nombre,
                  :lecturas_hora, :prom :ind_prom, :minimo :ind_min, :maximo :ind_max, :alertas
             FROM sensores s
             JOIN dispositivos d ON d.id = s.dispositivo_id
             LEFT JOIN lecturas l ON l.sensor_id = s.id AND l.medido_en > NOW() - INTERVAL '1
hour'
             WHERE s.codigo = :codigo
             GROUP BY s.id, s.tipo, s.unidad, s.umbral_max, d.codigo, d.nombre;
    if (fallo("consultar el sensor")) return 1;
    if (sqlca.sqlcode == ECPG_NOT_FOUND) {
        printf("El sensor %s no existe.\n", codigo);
        EXEC SQL COMMIT;
        return 1;
    }
    EXEC SQL COMMIT;
```

```
    printf("Sensor        : %s (%s, %s)\n", codigo, tipo, unidad);
    printf("Dispositivo   : %s - %s\n", disp, nombre);
    printf("Umbral maximo : %.2f %s\n", umbral, unidad);
    printf("Ultima hora   : %ld lecturas\n", lecturas_hora);
    if (ind_prom < 0)
        printf("Promedio      : sin lecturas en la ultima hora (NULL)\n");
    else
        printf("Promedio      : %.2f   minimo %.2f   maximo %.2f\n", prom, minimo, maximo);
    printf("Alertas pendientes: %ld\n", alertas);
    return 0;
}

/* lecturas <sensor> [n]: ultimas n lecturas (cursor con parametros) */
static int cmd_lecturas(const char *cod, int cuantas)
{
    EXEC SQL BEGIN DECLARE SECTION;
    char codigo[21], hora[20], severidad[13];
    long id;
    double valor;
    int limite;
    short ind_sev;
    EXEC SQL END DECLARE SECTION;
    int n = 0;

    snprintf(codigo, sizeof codigo, "%s", cod);
    limite = cuantas > 0 ? cuantas : 10;

    EXEC SQL DECLARE cur_lecturas CURSOR FOR
        SELECT l.id, l.valor, to_char(l.medido_en, 'YYYY-MM-DD HH24:MI:SS'), a.severidad
        FROM lecturas l
        JOIN sensores s ON s.id = l.sensor_id
        LEFT JOIN alertas a ON a.lectura_id = l.id
        WHERE s.codigo = :codigo
        ORDER BY l.medido_en DESC
        LIMIT :limite;
    EXEC SQL OPEN cur_lecturas;
    if (fallo("abrir el cursor")) return 1;

    printf("Ultimas %d lecturas de %s\n%-8s %10s  %-19s  %s\n", limite, codigo,
           "ID", "VALOR", "FECHA", "ALERTA");
    for (;;) {
        EXEC SQL FETCH cur_lecturas INTO :id, :valor, :hora, :severidad :ind_sev;
        if (sqlca.sqlcode == ECPG_NOT_FOUND || sqlca.sqlcode < 0) break;
        printf("%-8ld %10.2f  %-19s  %s\n", id, valor, hora, ind_sev < 0 ? "-" : severidad);
        n++;
    }
    EXEC SQL CLOSE cur_lecturas;
    EXEC SQL COMMIT;
    if (n == 0) printf("(sin lecturas: revisa el codigo del sensor)\n");
    return 0;
}

/* resumen: indicadores generales y promedios por tipo (agregados) */
static int cmd_resumen(void)
{
    EXEC SQL BEGIN DECLARE SECTION;
    long total, ultimo_min, pendientes, criticas;
    char tipo[21];
    double prom;
    long n;
    EXEC SQL END DECLARE SECTION;

    EXEC SQL SELECT (SELECT COUNT(*) FROM lecturas),
                    (SELECT COUNT(*) FROM lecturas WHERE medido_en > NOW() - INTERVAL '1 minute'),
                    COUNT(*) FILTER (WHERE NOT atendida),
                    COUNT(*) FILTER (WHERE NOT atendida AND severidad = 'critica')
             INTO :total, :ultimo_min, :pendientes, :criticas
             FROM alertas;
```

```
    if (fallo("calcular el resumen")) return 1;

    printf("Lecturas totales      : %ld\n", total);
    printf("Lecturas ultimo minuto: %ld\n", ultimo_min);
    printf("Alertas pendientes    : %ld (%ld criticas)\n\n", pendientes, criticas);

    EXEC SQL DECLARE cur_tipos CURSOR FOR
        SELECT s.tipo, ROUND(AVG(l.valor), 2), COUNT(*)
        FROM lecturas l JOIN sensores s ON s.id = l.sensor_id
        WHERE l.medido_en > NOW() - INTERVAL '10 minutes'
        GROUP BY s.tipo ORDER BY s.tipo;
    EXEC SQL OPEN cur_tipos;
    printf("Promedio de los ultimos 10 minutos por tipo:\n");
    for (;;) {
        EXEC SQL FETCH cur_tipos INTO :tipo, :prom, :n;
        if (sqlca.sqlcode == ECPG_NOT_FOUND || sqlca.sqlcode < 0) break;
        printf("  %-12s %8.2f  (%ld lecturas)\n", tipo, prom, n);
    }
    EXEC SQL CLOSE cur_tipos;
    EXEC SQL COMMIT;
    return 0;
}

/* monitor [segundos]: resumen que se actualiza cada 2 s (tiempo real) */
static int cmd_monitor(int segundos)
{
    time_t fin = time(NULL) + (segundos > 0 ? segundos : 30);
    while (time(NULL) < fin) {
        printf("\033[H\033[J== IoT Analytics Dashboard · monitor en C (Ctrl+C para salir) ==\n\
n");
        if (cmd_resumen() != 0) return 1;
        fflush(stdout);
        sleep(2);
    }
    return 0;
}

/* ========================================================================= */
/* C · CREAR                                                                 */
/* ========================================================================= */

/* crear-sensor <codigo> <dispositivo> <tipo> <unidad> <umbral> */
static int cmd_crear_sensor(char **a)
{
    EXEC SQL BEGIN DECLARE SECTION;
    char codigo[21], disp[21], tipo[21], unidad[11];
    double umbral;
    int id;
    EXEC SQL END DECLARE SECTION;

    snprintf(codigo, sizeof codigo, "%s", a[0]);
    snprintf(disp, sizeof disp, "%s", a[1]);
    snprintf(tipo, sizeof tipo, "%s", a[2]);
    snprintf(unidad, sizeof unidad, "%s", a[3]);
    umbral = atof(a[4]);

    EXEC SQL INSERT INTO sensores (dispositivo_id, codigo, tipo, unidad, umbral_max)
             SELECT d.id, :codigo, :tipo, :unidad, :umbral
             FROM dispositivos d WHERE d.codigo = :disp
             RETURNING id INTO :id;
    if (fallo("crear el sensor")) return 1;
    if (sqlca.sqlcode == ECPG_NOT_FOUND) {
        printf("El dispositivo %s no existe; no se creo el sensor.\n", disp);
        EXEC SQL ROLLBACK;
        return 1;
    }
    EXEC SQL COMMIT;
```

```
    printf("Sensor %s creado con id %d en %s (umbral %.2f %s).\n", codigo, id, disp, umbral,
unidad);
    return 0;
}

/* registrar <sensor> <valor>: inserta una lectura y muestra si genero alerta */
static int cmd_registrar(const char *cod, double v)
{
    EXEC SQL BEGIN DECLARE SECTION;
    char codigo[21], severidad[13];
    double valor;
    long id;
    short ind_sev;
    EXEC SQL END DECLARE SECTION;

    snprintf(codigo, sizeof codigo, "%s", cod);
    valor = v;
    EXEC SQL INSERT INTO lecturas (sensor_id, valor)
             SELECT id, :valor FROM sensores WHERE codigo = :codigo
             RETURNING id INTO :id;
    if (fallo("registrar la lectura")) return 1;
    if (sqlca.sqlcode == ECPG_NOT_FOUND) {
        printf("El sensor %s no existe.\n", codigo);
        EXEC SQL ROLLBACK;
        return 1;
    }
    /* la alerta la crea el disparador de la base de datos en la misma transaccion */
    EXEC SQL SELECT MAX(severidad) INTO :severidad :ind_sev FROM alertas WHERE lectura_id = :id;
    EXEC SQL COMMIT;
    printf("Lectura %ld registrada: %s = %.2f  ->  %s\n", id, codigo, valor,
           ind_sev < 0 ? "normal" : severidad);
    return 0;
}

/* simular [segundos]: 9 lecturas por segundo con una sentencia preparada */
#define MAX_SENSORES 32
static int cmd_simular(int segundos)
{
    EXEC SQL BEGIN DECLARE SECTION;
    char codigo[21], severidad[13];
    double umbral, valor;
    long id;
    short ind_sev;
    const char *sql_insert = "INSERT INTO lecturas (sensor_id, valor) "
                             "SELECT id, ? FROM sensores WHERE codigo = ? RETURNING id";
    EXEC SQL END DECLARE SECTION;
    char codigos[MAX_SENSORES][21];
    double umbrales[MAX_SENSORES];
    int n = 0, ronda = 0;
    long total = 0, alertas = 0;
    time_t fin = time(NULL) + segundos;

    /* 1. cargar la lista de sensores una sola vez */
    EXEC SQL DECLARE cur_sim CURSOR FOR SELECT codigo, umbral_max FROM sensores ORDER BY id;
    EXEC SQL OPEN cur_sim;
    while (n < MAX_SENSORES) {
        EXEC SQL FETCH cur_sim INTO :codigo, :umbral;
        if (sqlca.sqlcode == ECPG_NOT_FOUND || sqlca.sqlcode < 0) break;
        strcpy(codigos[n], codigo);
        umbrales[n++] = umbral;
    }
    EXEC SQL CLOSE cur_sim;

    /* 2. preparar la sentencia: el servidor la analiza una vez y se ejecuta muchas */
    EXEC SQL PREPARE ins_lectura FROM :sql_insert;
    if (fallo("preparar la sentencia")) return 1;
    srand((unsigned) time(NULL));
    printf("Simulando %d sensores%s...\n", n, segundos > 0 ? "" : " sin limite de tiempo");
```

```
    while (segundos <= 0 || time(NULL) < fin) {
        int alertas_ronda = 0;
        ronda++;
        for (int i = 0; i < n; i++) {
            double r = (double) rand() / RAND_MAX;
            valor = umbrales[i] * (0.76 + 0.08 * ((double) rand() / RAND_MAX));
            if (r < 0.02)      valor = umbrales[i] * 1.10;   /* pico: advertencia */
            else if (r < 0.03) valor = umbrales[i] * 1.30;   /* pico: critica */
            strcpy(codigo, codigos[i]);
            EXEC SQL EXECUTE ins_lectura INTO :id USING :valor, :codigo;
            if (sqlca.sqlcode < 0) break;
            EXEC SQL SELECT MAX(severidad) INTO :severidad :ind_sev FROM alertas WHERE lectura_id
= :id;
            if (ind_sev >= 0) {
                printf("  [%s] %s = %.2f (umbral %.2f)\n", severidad, codigo, valor, umbrales[i]);
                alertas_ronda++;
            }
            total++;
        }
        if (fallo("registrar la ronda")) return 1;
        EXEC SQL COMMIT;                    /* una transaccion por ronda */
        alertas += alertas_ronda;
        if (ronda % 10 == 0)
            printf("Ronda %d: %ld lecturas registradas, %ld alertas\n", ronda, total, alertas);
        fflush(stdout);
        sleep(1);
    }
    EXEC SQL DEALLOCATE PREPARE ins_lectura;
    printf("Fin de la simulacion: %ld lecturas, %ld alertas\n", total, alertas);
    return 0;
}

/* ================================================================== */
/* U · ACTUALIZAR                                                     */
/* ================================================================== */

/* umbral <sensor> <nuevo>: actualiza el umbral mostrando el valor anterior */
static int cmd_umbral(const char *cod, double nuevo)
{
    EXEC SQL BEGIN DECLARE SECTION;
    char codigo[21];
    double umbral, anterior;
    EXEC SQL END DECLARE SECTION;

    snprintf(codigo, sizeof codigo, "%s", cod);
    umbral = nuevo;
    EXEC SQL SELECT umbral_max INTO :anterior FROM sensores WHERE codigo = :codigo FOR UPDATE;
    if (fallo("leer el umbral")) return 1;
    if (sqlca.sqlcode == ECPG_NOT_FOUND) {
        printf("El sensor %s no existe.\n", codigo);
        EXEC SQL ROLLBACK;
        return 1;
    }
    EXEC SQL UPDATE sensores SET umbral_max = :umbral WHERE codigo = :codigo;
    if (fallo("actualizar el umbral")) return 1;
    long filas = sqlca.sqlerrd[2];          /* leer antes del COMMIT, que reinicia sqlca */
    EXEC SQL COMMIT;
    printf("Umbral de %s actualizado: %.2f -> %.2f (%ld fila)\n",
           codigo, anterior, umbral, filas);
    return 0;
}

/* estado <dispositivo> <activo|mantenimiento|inactivo> */
static int cmd_estado(const char *cod, const char *nuevo)
{
    EXEC SQL BEGIN DECLARE SECTION;
    char codigo[21], estado[16];
```

```
    EXEC SQL END DECLARE SECTION;

    snprintf(codigo, sizeof codigo, "%s", cod);
    snprintf(estado, sizeof estado, "%s", nuevo);
    EXEC SQL UPDATE dispositivos SET estado = :estado WHERE codigo = :codigo;
    if (fallo("cambiar el estado")) return 1;
    if (sqlca.sqlerrd[2] == 0) {            /* filas afectadas */
        printf("El dispositivo %s no existe.\n", codigo);
        EXEC SQL ROLLBACK;
        return 1;
    }
    EXEC SQL COMMIT;
    printf("Dispositivo %s ahora esta en %s.\n", codigo, estado);
    return 0;
}

/* ================================================================== */
/* D · ELIMINAR                                                       */
/* ================================================================== */

/* eliminar-lectura <id>: sus alertas se borran en cascada */
static int cmd_eliminar_lectura(long lid)
{
    EXEC SQL BEGIN DECLARE SECTION;
    long id;
    EXEC SQL END DECLARE SECTION;

    id = lid;
    EXEC SQL DELETE FROM lecturas WHERE id = :id;
    if (fallo("eliminar la lectura")) return 1;
    if (sqlca.sqlerrd[2] == 0) {
        printf("La lectura %ld no existe.\n", id);
        EXEC SQL ROLLBACK;
        return 1;
    }
    EXEC SQL COMMIT;
    printf("Lectura %ld eliminada.\n", id);
    return 0;
}

/* eliminar-sensor <codigo> [--con-lecturas]
 * Sin la opcion, PostgreSQL rechaza el borrado si el sensor tiene lecturas (23503).
 * Con la opcion, borra alertas, lecturas y sensor en UNA transaccion: todo o nada. */
static int cmd_eliminar_sensor(const char *cod, int con_lecturas)
{
    EXEC SQL BEGIN DECLARE SECTION;
    char codigo[21];
    long n_alertas = 0, n_lecturas = 0;
    EXEC SQL END DECLARE SECTION;

    snprintf(codigo, sizeof codigo, "%s", cod);
    if (con_lecturas) {
        EXEC SQL DELETE FROM alertas WHERE sensor_id = (SELECT id FROM sensores WHERE codigo
= :codigo);
        n_alertas = sqlca.sqlerrd[2];
        EXEC SQL DELETE FROM lecturas WHERE sensor_id = (SELECT id FROM sensores WHERE codigo
= :codigo);
        n_lecturas = sqlca.sqlerrd[2];
    }
    EXEC SQL DELETE FROM sensores WHERE codigo = :codigo;
    if (fallo("eliminar el sensor")) {
        if (!con_lecturas)
            fprintf(stderr, "Usa: eliminar-sensor %s --con-lecturas\n", codigo);
        return 1;
    }
    if (sqlca.sqlerrd[2] == 0) {
        printf("El sensor %s no existe.\n", codigo);
        EXEC SQL ROLLBACK;
```

```
        return 1;
    }
    EXEC SQL COMMIT;
    printf("Sensor %s eliminado (%ld lecturas y %ld alertas en la misma transaccion).\n",
           codigo, n_lecturas, n_alertas);
    return 0;
}

/* ================================================================ */
/* Analisis de rendimiento                                          */
/* ================================================================ */
static double ahora_ms(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec * 1000.0 + t.tv_nsec / 1e6;
}

/* benchmark [n]: inserta n lecturas normales de tres formas y las borra al final */
static int cmd_benchmark(int n)
{
    EXEC SQL BEGIN DECLARE SECTION;
    char sql[256];
    double valor;
    long desde;
    const char *sql_prep = "INSERT INTO lecturas (sensor_id, valor) "
                           "SELECT id, ? FROM sensores WHERE codigo = 'PRE03-VIB'";
    EXEC SQL END DECLARE SECTION;
    double t0, t1, t2, t3;

    if (n <= 0) n = 1000;
    EXEC SQL SELECT COALESCE(MAX(id), 0) INTO :desde FROM lecturas;
    EXEC SQL COMMIT;
    printf("Insertando %d lecturas de PRE03-VIB de tres formas...\n\n", n);

    /* A. SQL dinamico (texto armado con sprintf) y COMMIT por fila */
    t0 = ahora_ms();
    for (int i = 0; i < n; i++) {
        snprintf(sql, sizeof sql, "INSERT INTO lecturas (sensor_id, valor) "
                 "SELECT id, %.2f FROM sensores WHERE codigo = 'PRE03-VIB'", 5.0 + (i % 10) *
0.1);
        EXEC SQL EXECUTE IMMEDIATE :sql;
        EXEC SQL COMMIT;
    }
    t1 = ahora_ms();

    /* B. Sentencia preparada y COMMIT por fila */
    EXEC SQL PREPARE ins_bench FROM :sql_prep;
    for (int i = 0; i < n; i++) {
        valor = 5.0 + (i % 10) * 0.1;
        EXEC SQL EXECUTE ins_bench USING :valor;
        EXEC SQL COMMIT;
    }
    t2 = ahora_ms();

    /* C. Sentencia preparada y UNA sola transaccion */
    for (int i = 0; i < n; i++) {
        valor = 5.0 + (i % 10) * 0.1;
        EXEC SQL EXECUTE ins_bench USING :valor;
    }
    EXEC SQL COMMIT;
    t3 = ahora_ms();
    EXEC SQL DEALLOCATE PREPARE ins_bench;
    if (fallo("ejecutar la prueba")) return 1;

    printf("%-48s %10s %12s\n", "FORMA", "TOTAL", "POR LECTURA");
    printf("%-48s %8.1f ms %9.3f ms\n", "A. SQL dinamico + COMMIT por fila", t1 - t0, (t1 - t0) /
n);
```

```
    printf("%-48s %8.1f ms %9.3f ms\n", "B. Sentencia preparada + COMMIT por fila", t2 - t1, (t2 -
t1) / n);
    printf("%-48s %8.1f ms %9.3f ms\n", "C. Sentencia preparada + una transaccion", t3 - t2, (t3 -
t2) / n);
    printf("\nC fue %.1f veces mas rapida que A.\n", (t1 - t0) / (t3 - t2));

    EXEC SQL DELETE FROM lecturas WHERE id > :desde
             AND sensor_id = (SELECT id FROM sensores WHERE codigo = 'PRE03-VIB');
    printf("Limpieza: %ld lecturas de prueba eliminadas.\n", sqlca.sqlerrd[2]);
    EXEC SQL COMMIT;
    return 0;
}

/* ------------------------------------------------------------------ */
static void ayuda(void)
{
    puts("Uso: iot_crud <comando> [argumentos]\n\n"
         "  CREAR\n"
         "    crear-sensor <codigo> <dispositivo> <tipo> <unidad> <umbral>\n"
         "    registrar <sensor> <valor>\n"
         "    simular [segundos]            (0 = sin limite)\n"
         "  LEER\n"
         "    listar\n"
         "    sensor <codigo>\n"
         "    lecturas <sensor> [cantidad]\n"
         "    resumen\n"
         "    monitor [segundos]\n"
         "  ACTUALIZAR\n"
         "    umbral <sensor> <nuevo>\n"
         "    estado <dispositivo> <activo|mantenimiento|inactivo>\n"
         "  ELIMINAR\n"
         "    eliminar-lectura <id>\n"
         "    eliminar-sensor <codigo> [--con-lecturas]\n"
         "  RENDIMIENTO\n"
         "    benchmark [cantidad]");
}

int main(int argc, char **argv)
{
    int rc;
    const char *c = argc > 1 ? argv[1] : "ayuda";

    /* Sin terminal (docker exec, logs) stdout usa un bufer y se mezcla en desorden con
       stderr; con bufer por linea, cada mensaje sale en el orden en que se imprime */
    setvbuf(stdout, NULL, _IOLBF, 0);

    if (strcmp(c, "ayuda") == 0 || strcmp(c, "--help") == 0) { ayuda(); return 0; }
    if (conectar() != 0) return 2;

    if      (!strcmp(c, "listar"))                          rc = cmd_listar();
    else if (!strcmp(c, "sensor") && argc > 2)              rc = cmd_sensor(argv[2]);
    else if (!strcmp(c, "lecturas") && argc > 2)            rc = cmd_lecturas(argv[2], argc > 3 ?
atoi(argv[3]) : 10);
    else if (!strcmp(c, "resumen"))                         rc = cmd_resumen();
    else if (!strcmp(c, "monitor"))                         rc = cmd_monitor(argc > 2 ?
atoi(argv[2]) : 30);
    else if (!strcmp(c, "crear-sensor") && argc > 6)        rc = cmd_crear_sensor(argv + 2);
    else if (!strcmp(c, "registrar") && argc > 3)           rc = cmd_registrar(argv[2],
atof(argv[3]));
    else if (!strcmp(c, "simular"))                         rc = cmd_simular(argc > 2 ?
atoi(argv[2]) : 60);
    else if (!strcmp(c, "umbral") && argc > 3)              rc = cmd_umbral(argv[2],
atof(argv[3]));
    else if (!strcmp(c, "estado") && argc > 3)              rc = cmd_estado(argv[2], argv[3]);
    else if (!strcmp(c, "eliminar-lectura") && argc > 2)    rc =
cmd_eliminar_lectura(atol(argv[2]));
    else if (!strcmp(c, "eliminar-sensor") && argc > 2)     rc = cmd_eliminar_sensor(argv[2],
```

```
                                                        argc > 3 && !strcmp(argv[3], "--
con-lecturas"));
    else if (!strcmp(c, "benchmark"))                    rc = cmd_benchmark(argc > 2 ?
atoi(argv[2]) : 1000);
    else { ayuda(); rc = 1; }

    EXEC SQL DISCONNECT iot;
    return rc ? 1 : 0;
}
```

4.4 Base de datos: init.sql

```
-- ========================================================================
-- TAREA 10 · Programacion del lado del cliente: SQL embebido con C
-- IoT Analytics Dashboard (NexoDev Solutions) · PostgreSQL 16
-- Base de datos que el programa en C (ECPG) administra con operaciones CRUD.
-- ========================================================================

CREATE TABLE dispositivos (
    id        SERIAL PRIMARY KEY,
    codigo    VARCHAR(20) NOT NULL UNIQUE,
    nombre    VARCHAR(60) NOT NULL,
    ubicacion VARCHAR(60) NOT NULL,
    estado    VARCHAR(15) NOT NULL DEFAULT 'activo'
              CHECK (estado IN ('activo', 'mantenimiento', 'inactivo'))
);

CREATE TABLE sensores (
    id             SERIAL PRIMARY KEY,
    dispositivo_id INT NOT NULL REFERENCES dispositivos(id),
    codigo         VARCHAR(20) NOT NULL UNIQUE,
    tipo           VARCHAR(20) NOT NULL CHECK (tipo IN ('temperatura', 'vibracion', 'energia',
'humedad')),
    unidad         VARCHAR(10) NOT NULL,
    umbral_max     NUMERIC(10,2) NOT NULL CHECK (umbral_max > 0)
);

CREATE TABLE lecturas (
    id        BIGSERIAL PRIMARY KEY,
    sensor_id INT NOT NULL REFERENCES sensores(id),
    valor     NUMERIC(10,2) NOT NULL,
    medido_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_lecturas_sensor_fecha ON lecturas (sensor_id, medido_en DESC);
CREATE INDEX idx_lecturas_fecha        ON lecturas (medido_en DESC);

CREATE TABLE alertas (
    id         SERIAL PRIMARY KEY,
    lectura_id BIGINT NOT NULL REFERENCES lecturas(id) ON DELETE CASCADE,
    sensor_id  INT NOT NULL REFERENCES sensores(id),
    severidad  VARCHAR(12) NOT NULL CHECK (severidad IN ('advertencia', 'critica')),
    valor      NUMERIC(10,2) NOT NULL,
    umbral     NUMERIC(10,2) NOT NULL,
    creada_en  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    atendida   BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE INDEX idx_alertas_lectura ON alertas (lectura_id);

-- Alerta automatica (de la Tarea 9): el programa en C solo inserta la lectura
CREATE FUNCTION trg_lecturas_alerta() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
    v_umbral NUMERIC;
BEGIN
    SELECT umbral_max INTO v_umbral FROM sensores WHERE id = NEW.sensor_id;
    IF NEW.valor > v_umbral THEN
        INSERT INTO alertas (lectura_id, sensor_id, severidad, valor, umbral, creada_en)
```

```
       VALUES (NEW.id, NEW.sensor_id,
               CASE WHEN NEW.valor > v_umbral * 1.2 THEN 'critica' ELSE 'advertencia' END,
               NEW.valor, v_umbral, NEW.medido_en);
    END IF;
    RETURN NULL;
END $$;

CREATE TRIGGER t_lecturas_alerta AFTER INSERT ON lecturas
FOR EACH ROW EXECUTE FUNCTION trg_lecturas_alerta();

-- Usuario del programa en C: CRUD sobre las tablas, sin poder cambiar la estructura
CREATE ROLE app_iot LOGIN PASSWORD 'app123';
GRANT SELECT, INSERT, UPDATE, DELETE ON dispositivos, sensores, lecturas, alertas TO app_iot;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO app_iot;

-- Datos iniciales
INSERT INTO dispositivos (codigo, nombre, ubicacion) VALUES
('PRE-03', 'Prensa hidraulica 3', 'Linea 1'),
('TOR-01', 'Torno CNC 1',        'Linea 2'),
('EMB-02', 'Embotelladora 2',    'Linea 3');

INSERT INTO sensores (dispositivo_id, codigo, tipo, unidad, umbral_max) VALUES
(1, 'PRE03-TEMP', 'temperatura', '°C',   85.00),
(1, 'PRE03-VIB',  'vibracion',   'mm/s',  7.00),
(1, 'PRE03-ENE',  'energia',     'kWh',  30.00),
(2, 'TOR01-TEMP', 'temperatura', '°C',   80.00),
(2, 'TOR01-VIB',  'vibracion',   'mm/s',  6.00),
(2, 'TOR01-ENE',  'energia',     'kWh',  25.00),
(3, 'EMB02-TEMP', 'temperatura', '°C',   75.00),
(3, 'EMB02-VIB',  'vibracion',   'mm/s',  5.00),
(3, 'EMB02-ENE',  'energia',     'kWh',  20.00);

-- 6 horas de historico (una lectura cada 30 s por sensor)
SELECT setseed(0.5);
INSERT INTO lecturas (sensor_id, valor, medido_en)
SELECT s.id, ROUND((s.umbral_max * (0.80 + (random() - 0.5) * 0.08))::numeric, 2), t
FROM sensores s
CROSS JOIN generate_series(NOW() - INTERVAL '6 hours', NOW() - INTERVAL '1 minute',
                           INTERVAL '30 seconds') AS t;
ANALYZE;
```

4.5 Docker Compose

```
# Tarea 10 · SQL embebido con C · IoT Analytics Dashboard (NexoDev Solutions)
name: iot-sql-c

services:
  # ================= BASE DE DATOS =================
  datos:
    image: postgres:16
    container_name: sqlc-datos
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD:-admin123}
      POSTGRES_DB: iotanalytics
    command: >
      -c ssl=on -c ssl_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
      -c ssl_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
    volumes:
      - ./datos/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    networks: [red-datos]          # sin puerto expuesto a la PC
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d iotanalytics"]
      interval: 3s
      retries: 30

  # ========= CLIENTE EN C: se usa con "docker exec sqlc-cliente iot_crud ..." =========
  cliente:
```

```
    build: ./cliente
    container_name: sqlc-cliente
    environment: &conexion
      DB_HOST: datos
      DB_USER: app_iot
      DB_PASSWORD: app123
      DB_SSLMODE: require
      PGTZ: America/Mexico_City   # horas en la zona horaria local
    networks: [red-datos]
    depends_on:
      datos: { condition: service_healthy }

  # ========= SIMULADOR: el mismo programa en C enviando lecturas en tiempo real =========
  simulador:
    build: ./cliente
    container_name: sqlc-simulador
    command: ["iot_crud", "simular", "0"]
    environment: *conexion
    networks: [red-datos]
    depends_on:
      datos: { condition: service_healthy }

networks:
  red-datos:
```

5. Pruebas y ejemplos de uso

La base inicia con 6 horas de histórico (6,480 lecturas) y el simulador agrega 9 lecturas por segundo, con picos que generan alertas. Cada comando se ejecuta con `docker exec sqlc-cliente iot_crud <comando>`.

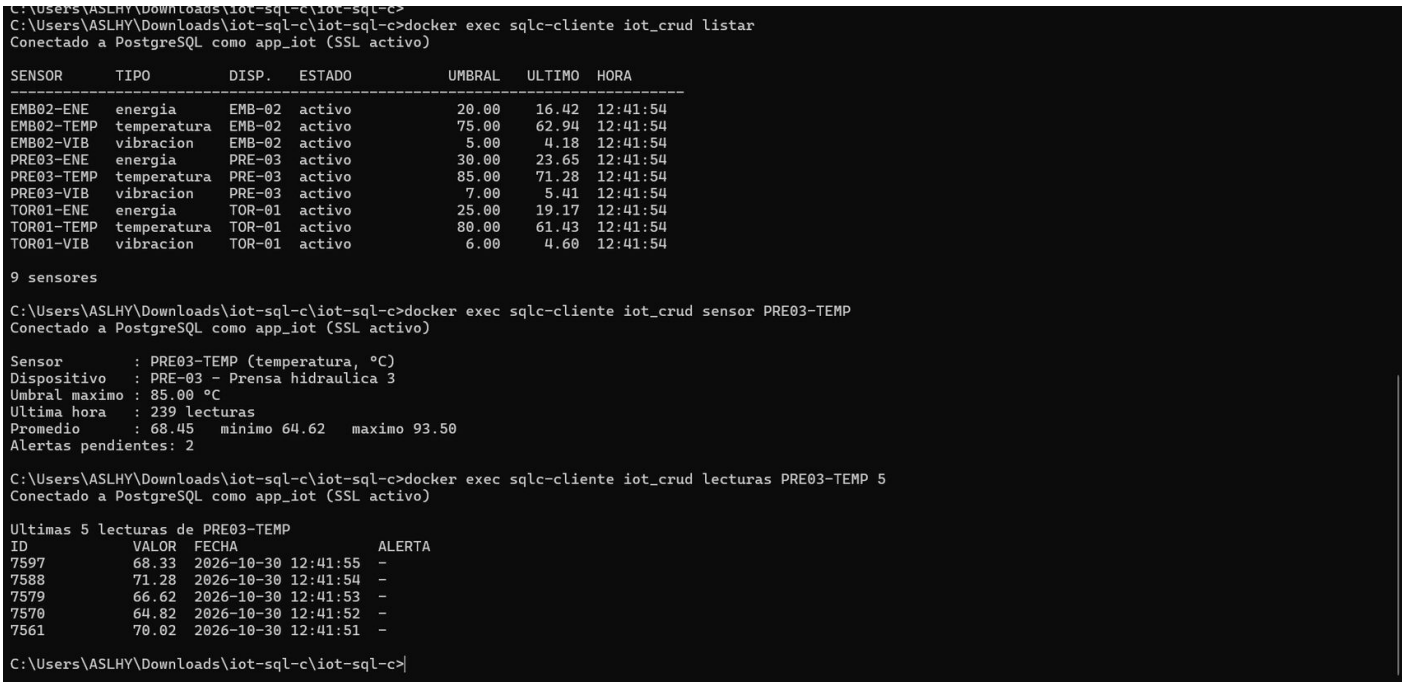
NO.	PRUEBA	RESULTADO ESPERADO	RESULTADO OBTENIDO
P1	Compilación y arranque	ecpg + gcc sin errores; 3 contenedores	Correcto: imagen compilada; base healthy; cliente y simulador iniciados
P2	Conexión	Conexión cifrada	Correcto: “Conectado a PostgreSQL como app_iot (SSL activo)”
P3	Leer (cursor e INTO)	Sensores, detalle y lecturas	Correcto: 9 sensores; PRE03-TEMP con 239 lecturas en la última hora y 2 alertas
P4	Crear	Sensor nuevo y lecturas	Correcto: EMB02-HUM con id 12; lectura de 90 % clasificada como crítica
P5	Actualizar	Umbral y estado	Correcto: 70.00 → 75.00 (1 fila); TOR-01 a mantenimiento y de vuelta a activo
P6	Eliminar con dependencias	Rechazo por llave foránea	Correcto: SQLSTATE 23503 y ROLLBACK
P7	Eliminar en una transacción	Sensor, lecturas y alertas juntos	Correcto: 1 lectura y 1 alerta eliminadas con el sensor
P8	Llave duplicada / CHECK	Error explicado y ROLLBACK	Correcto: 23505 y 23514
P9	Registro inexistente	Mensaje claro (NOT FOUND)	Correcto: “El sensor NOEXISTE no existe”
P10	Tiempo real	Lecturas y alertas continuas	Correcto: 2,700 lecturas y 68 alertas en 300 rondas; 522 lecturas en el último minuto
P11	Rendimiento	Una transacción más rápida que COMMIT por fila	Correcto: 22.5 veces más rápida

Evidencia 1 — P1: compilación dentro de Docker y arranque de los contenedores.



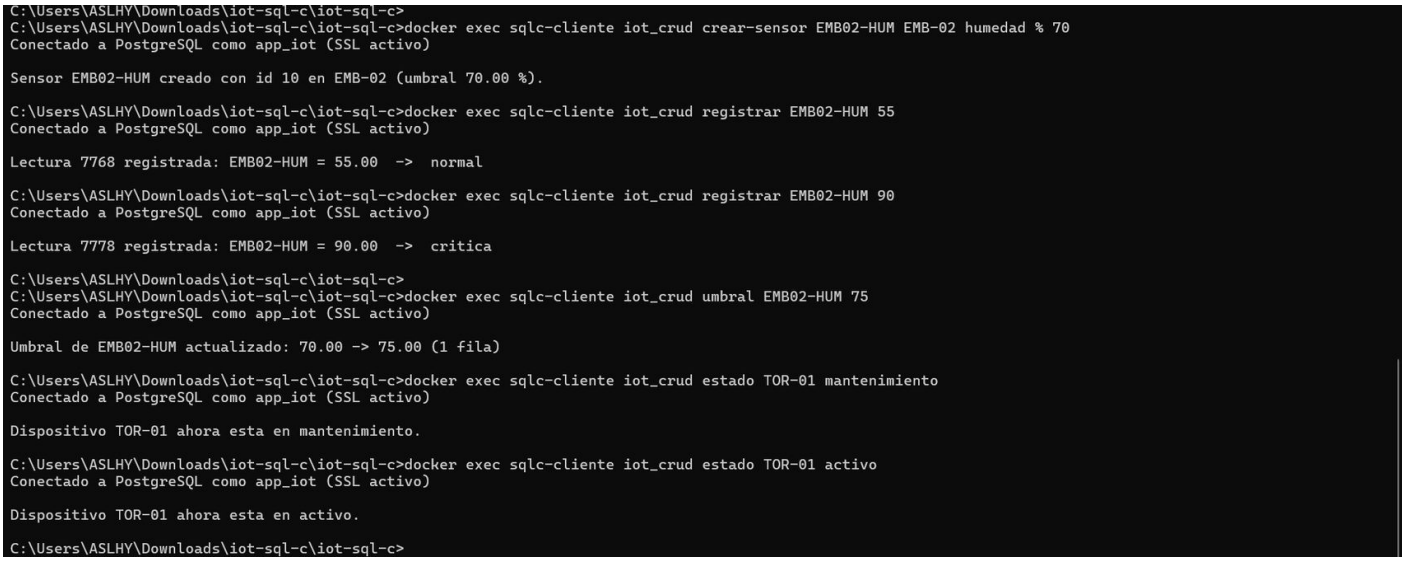
Arranque del entorno.

Evidencia 2 — P2 y P3 (Leer): el programa se conecta con SSL, lista los sensores con un cursor, consulta el detalle de un sensor con SELECT INTO y muestra las últimas lecturas.



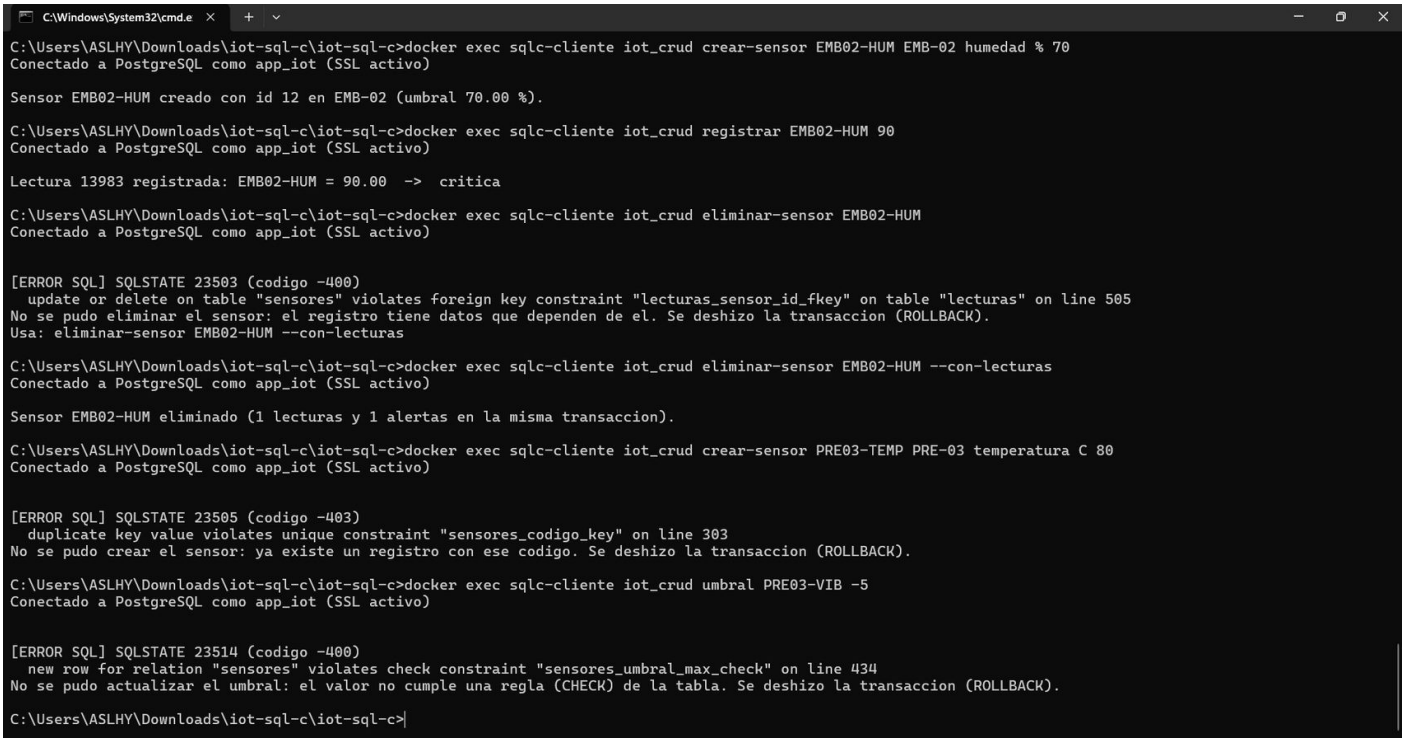
Operaciones de lectura.

Evidencia 3 — P4 y P5 (Crear y Actualizar): se crea el sensor EMB02-HUM, se registran dos lecturas (la de 90 % genera una alerta crítica mediante el disparador), se actualiza el umbral mostrando el valor anterior y se cambia el estado de TOR-01.



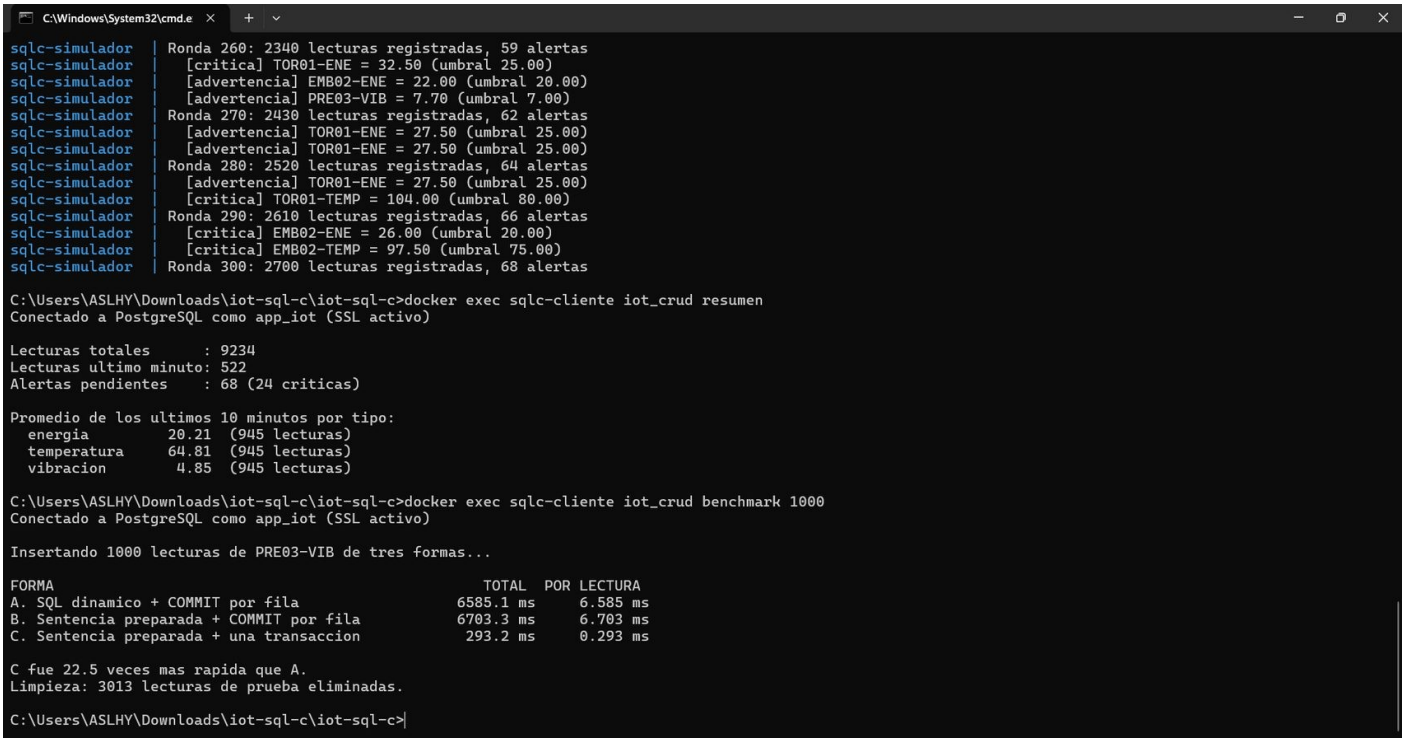
Operaciones de creación y actualización.

Evidencia 4 — P6 a P8 (Eliminar y errores): el borrado simple es rechazado por la llave foránea; con --con-lecturas se elimina todo en una transacción; los errores de llave duplicada y CHECK se explican y se deshacen.



Eliminación y manejo de errores.

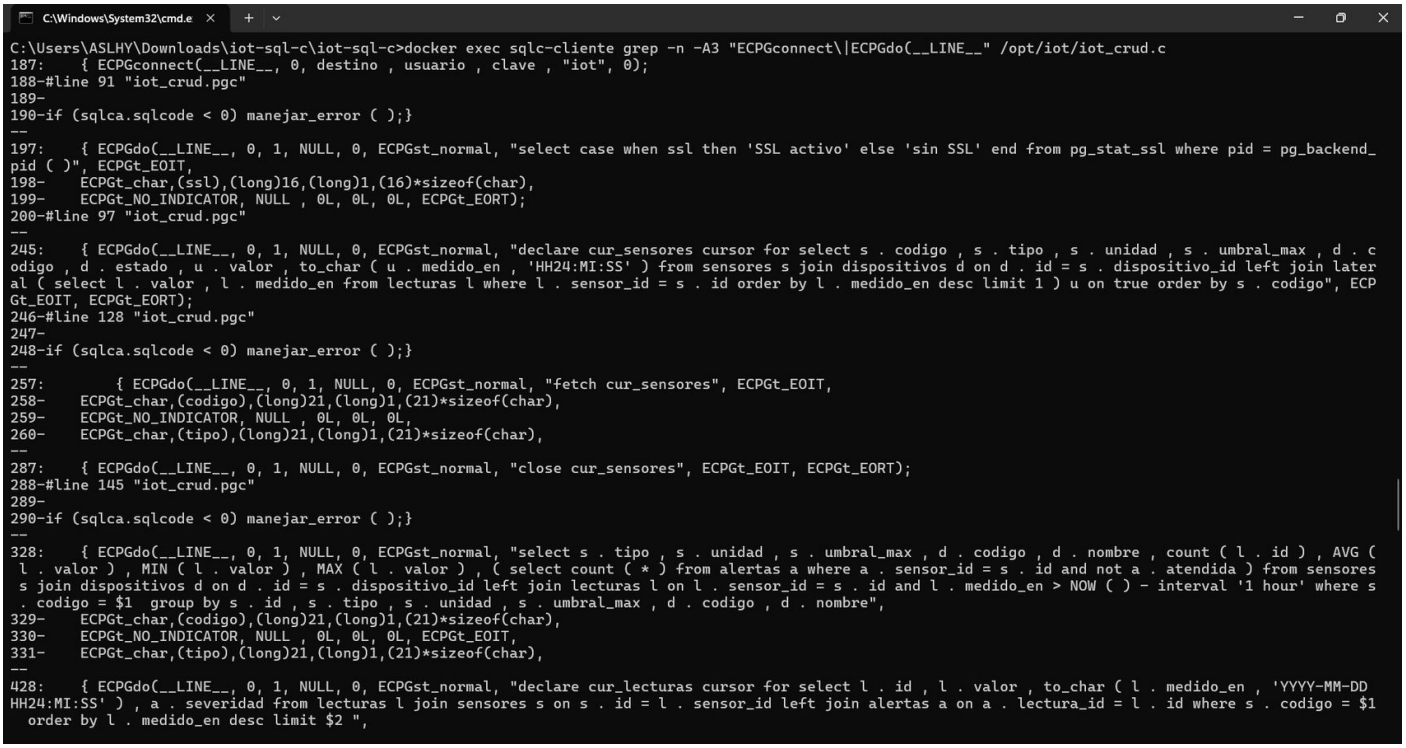
Evidencia 5 — P10 y P11: el simulador almacena lecturas en tiempo real; el resumen muestra 522 lecturas en el último minuto; y el benchmark compara tres formas de insertar.



Tiempo real y prueba de rendimiento.

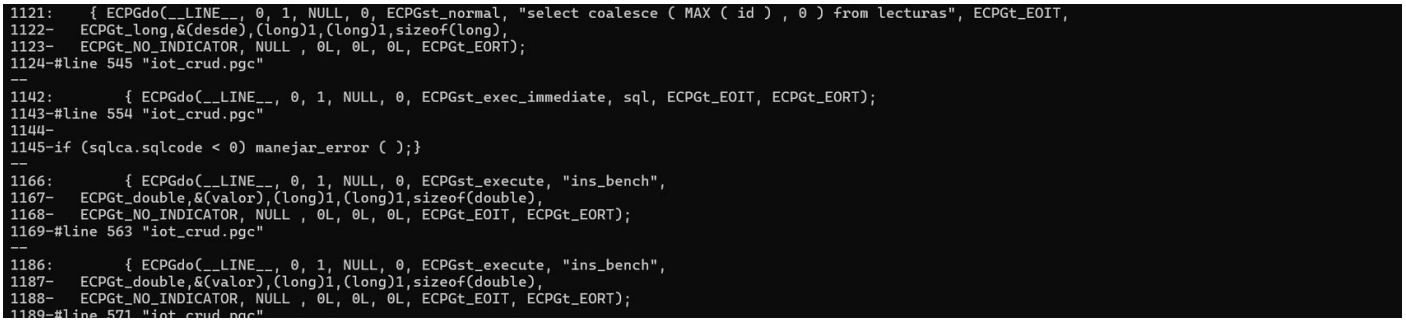
Evidencia 6: el código C que generó ECPG dentro del contenedor. EXEC SQL CONNECT se convirtió en ECPGconnect; cada consulta, en ECPGdo con el SQL como texto y las variables de enlace como parámetros (,) con su tipo y tamaño (ECPGt_char, ECPGt_long); y cada sentencia quedó seguida de la revisión if (sqlca.sqlcode < 0) manejar_error(), que es lo que produce

WHENEVER SQLERROR. Las líneas #line enlazan cada llamada con su línea original del archivo .pgc.



Código generado por ECPG: conexión, consulta y cursor.

Las dos formas del benchmark también se distinguen en el código generado: el SQL dinámico se ejecuta con ECPGst_exec_immediate a partir del texto armado con sprintf, mientras que la sentencia preparada se ejecuta con ECPGst_execute por su nombre (ins_bench), enviando solo el valor como parámetro.



SQL dinámico frente a sentencia preparada en el código generado.

6. Análisis de rendimiento

El comando `benchmark` inserta 1,000 lecturas de tres formas y después las elimina:

FORMA	EN LA PC (DOCKER DESKTOP)	EN EL EQUIPO DE DESARROLLO	POR LECTURA (PC)
A. SQL dinámico (EXECUTE IMMEDIATE) + COMMIT por fila	6,585.1 ms	650.3 ms	6.585 ms
B. Sentencia preparada + COMMIT por fila	6,703.3 ms	520.3 ms	6.703 ms
C. Sentencia preparada + una transacción	293.2 ms	103.4 ms	0.293 ms
Mejora de C sobre A	22.5 veces	6.3 veces	

Interpretación: en la PC, A y B tardan casi lo mismo. El costo dominante no es analizar el SQL, sino el **COMMIT**: cada uno obliga a PostgreSQL a escribir en disco el registro de la transacción, y en Docker Desktop sobre Windows cada escritura tarda cerca de 6.5 ms. Al agrupar las 1,000 inserciones en una transacción se hace una sola escritura y el tiempo cae 22.5 veces. En el equipo de desarrollo, con un disco más rápido, el COMMIT pesa menos y la ventaja de la sentencia preparada sí se nota (de 650 a 520 ms).

Decisiones que se tomaron por esta prueba: el simulador usa una sentencia preparada y hace un COMMIT por ronda de 9 lecturas, no uno por lectura; y el SQL dinámico solo se usa en el benchmark, porque además de ser más lento obliga a armar el texto con los valores, lo que abre la puerta a la inyección SQL si los datos vinieran del usuario.

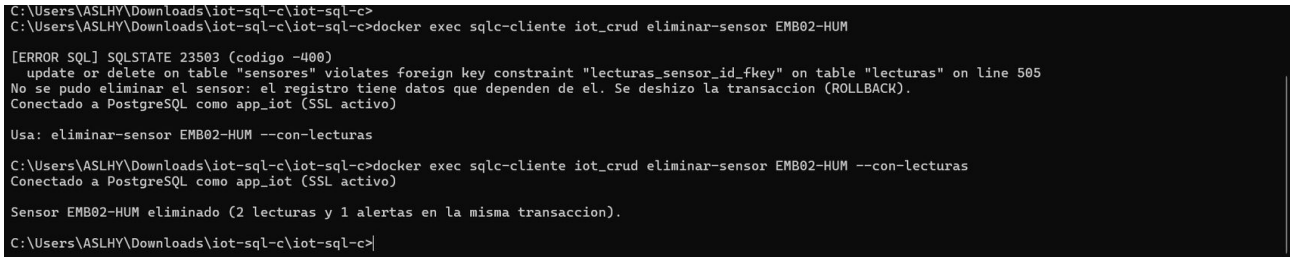
6.1 Impacto en el IoT Analytics Dashboard

ASPECTO	APORTE DEL MÓDULO EN C
Equipos IoT con pocos recursos	Un ejecutable compilado sin intérprete ni máquina virtual; la imagen final no incluye el compilador
Integridad	Cada operación es una transacción; ante cualquier error se hace ROLLBACK y no quedan datos a medias
Seguridad	Conexión SSL, contraseña en variable de entorno y valores enviados como parámetros
Tiempo real	El simulador almacena 9 lecturas por segundo y `monitor` las consulta cada 2 s
Eficiencia	Agrupar lecturas por transacción reduce hasta 22 veces el tiempo de escritura

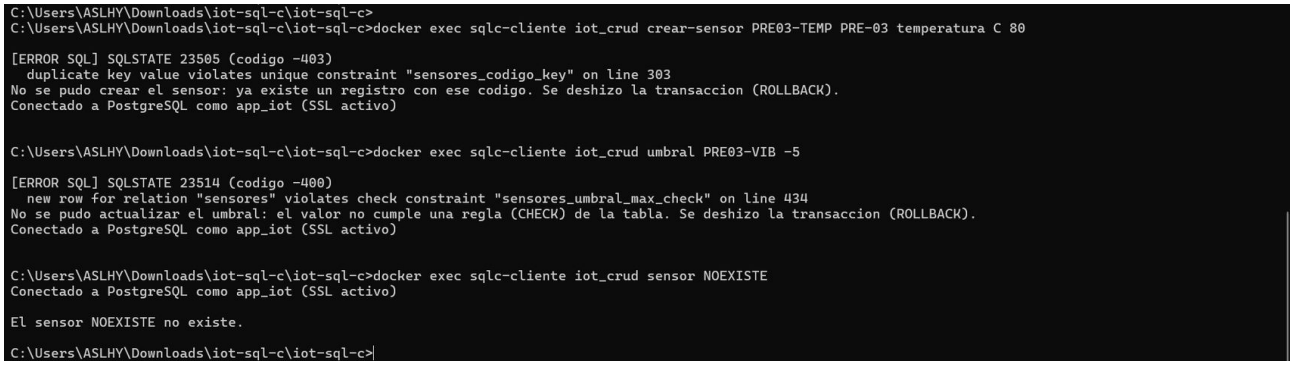
7. Problemas encontrados y solución

7.1 Mensajes en desorden al usar docker exec

En las primeras pruebas, cuando una operación fallaba, el mensaje “Conectado a PostgreSQL” aparecía **después** del error, aunque el programa lo imprime antes. La causa: sin una terminal, C guarda la salida normal (`stdout`) en un búfer y la escribe al final, mientras que los errores (`stderr`) salen de inmediato.



Antes: el mensaje de conexión sale después del error.



Antes: el mismo desorden en los errores de llave duplicada y CHECK.

Solución: al iniciar el programa se configura `stdout` con búfer por línea (`setvbuf(stdout, NULL, _IOLBF, 0)`), así cada mensaje sale en el orden en que se imprime. La Evidencia 4 muestra el resultado después de la corrección.

7.2 Contador de filas en cero

En las pruebas de desarrollo, el comando `umbral` informaba “0 fila” aunque sí actualizaba. El programa leía `sqlca.sqlerrd[2]` después del COMMIT, y el COMMIT es otra sentencia que reinicia `sqlca`. Se corrigió guardando el contador en una variable antes de confirmar.

7.3 Prevención

El cliente y el simulador se construyen como imágenes separadas desde el inicio, aplicando la solución del error de construcción simultánea de la Tarea 9.

8. Control de versiones y colaboración

La actividad se desarrolló de forma individual. El proyecto se publicó en un repositorio público con un README que explica cada comando y las técnicas de SQL embebido: https://github.com/luisgael-ml/iot-sql-c

```
git init
git config user.name "Luis Gael Monterrubio López"
git config user.email "monterrubiolopezluisgael@gmail.com"
git add .
git commit -m "SQL embebido en C (ECPG) para IoT Analytics Dashboard"
git branch -M main
git remote add origin https://github.com/luisgael-ml/iot-sql-c.git
git push -u origin main
```

Evidencia 7: el commit se registró a nombre de Luis Gael Monterrubio López y se publicó en GitHub.

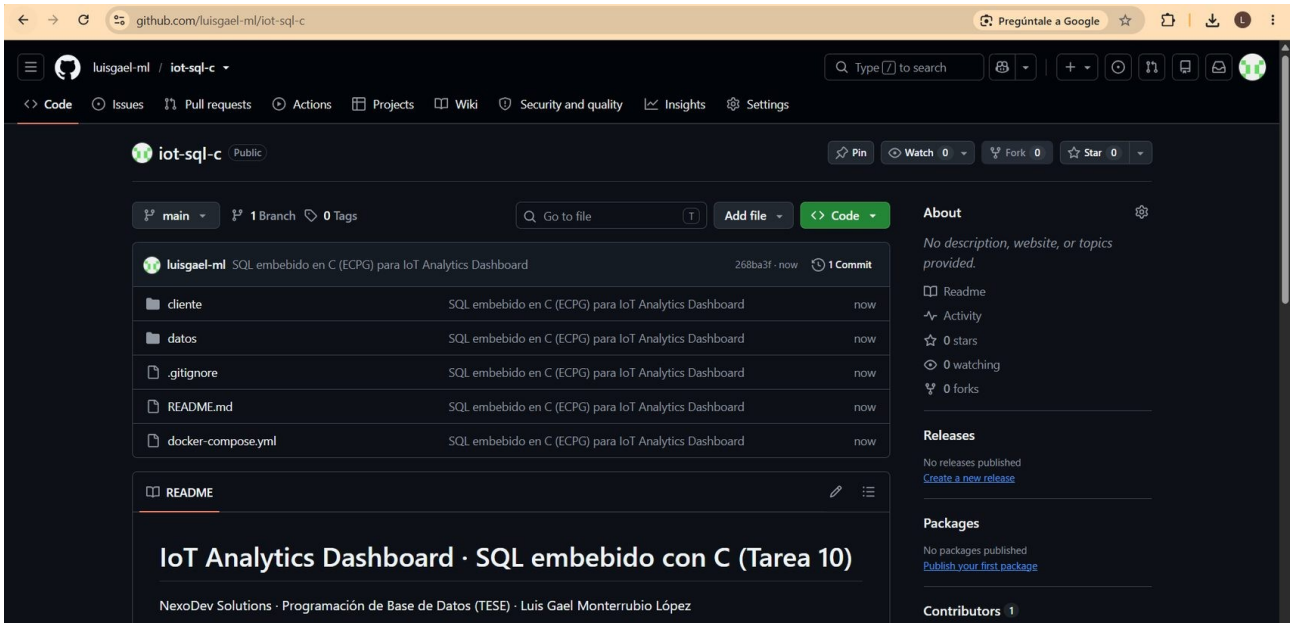
```
C:\Users\ASLHY\Downloads\iot-sql-c\iot-sql-c>git commit -m "SQL embebido en C (ECPG) para IoT Analytics Dashboard"
[main (root-commit) 268ba3f] SQL embebido en C (ECPG) para IoT Analytics Dashboard
 7 files changed, 859 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 cliente/Dockerfile
 create mode 100644 cliente/Makefile
 create mode 100644 cliente/iot_crud.pgc
 create mode 100644 datos/init.sql
 create mode 100644 docker-compose.yml

C:\Users\ASLHY\Downloads\iot-sql-c\iot-sql-c>git branch -M main

C:\Users\ASLHY\Downloads\iot-sql-c\iot-sql-c>git remote add origin https://github.com/luisgael-ml/iot-sql-c.git

C:\Users\ASLHY\Downloads\iot-sql-c\iot-sql-c>git push -u origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 4 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (11/11), 10.78 KiB | 1.54 MiB/s, done.
Total 11 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/luisgael-ml/iot-sql-c.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Publicación del proyecto en GitHub.



Repositorio iot-sql-c.

8.1 Reflexión individual sobre la colaboración

Al empezar busqué cómo se hace el SQL embebido real en PostgreSQL y encontré ECPG en la documentación oficial; preferí usarlo en lugar de la biblioteca libpq porque es lo que pide la competencia, aunque implicaba aprender el paso de precompilación. También retomé lo aprendido en entregas anteriores: la construcción de imágenes separadas de la Tarea 9 y el disparador de alertas, que aquí permitió que el programa en C se concentrara solo en el CRUD.

Para compartir el conocimiento con mis compañeros, preparé el proyecto para que cualquiera lo use sin instalar un compilador de C: Docker compila todo. El README resume los comandos y las técnicas, el comando `ayuda` muestra cómo usar cada operación, y documenté los dos errores que encontré, en especial el de los mensajes en desorden, que le puede pasar a cualquiera que ejecute programas en C dentro de contenedores. La comparación de rendimiento también sirve como guía práctica: agrupar operaciones en una transacción importa más que cualquier otra optimización.

9. Conclusiones

El SQL embebido permitió escribir las consultas tal como se escriben en SQL, dentro del programa en C, y dejar que ECPG se encargue de convertirlas en llamadas a la biblioteca. El programa cubre

el ciclo completo: se conecta con SSL, ejecuta consultas de una fila con `SELECT INTO` y de varias filas con cursores, recibe datos generados con `RETURNING`, detecta valores nulos con variables indicadoras y revisa cada resultado en `sqlca`.

Las operaciones CRUD sobre los sensores funcionaron con datos simulados y reales, y el manejo de errores convirtió cada violación de una restricción en un mensaje claro seguido de ROLLBACK, sin dejar datos a medias. El análisis de rendimiento mostró que agrupar las inserciones en una transacción fue 22.5 veces más rápido que confirmar cada una, una decisión que se aplicó al simulador. Así, el módulo almacena, consulta y actualiza los datos de los sensores en tiempo real de forma íntegra y eficiente, como lo necesita NexoDev Solutions.

Referencias

Docker Inc. (s.f.). *Multi-stage builds*. Docker Docs. https://docs.docker.com/build/building/multi-stage/

Free Software Foundation. (s.f.). *Buffering concepts (The GNU C Library)*. GNU. https://www.gnu.org/software/libc/manual/html_node/Buffering-Concepts.html

ISO/IEC. (s.f.). *ISO/IEC 9075-2: Information technology — Database languages — SQL*. ISO. https://www.iso.org/standard/76584.html

PostgreSQL Global Development Group. (s.f.). *ECPG — Embedded SQL in C*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/ecpg.html

PostgreSQL Global Development Group. (s.f.). *Error handling (ECPG)*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/ecpg-errors.html

PostgreSQL Global Development Group. (s.f.). *Managing database connections (ECPG)*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/ecpg-connect.html

PostgreSQL Global Development Group. (s.f.). *Using host variables (ECPG)*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/ecpg-variables.html

PostgreSQL Global Development Group. (s.f.). *WAL configuration*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/wal-configuration.html